

Từ Bài Toán Bàn Tròn Đến Việc Kết Hợp Cấu Trúc Dữ Liệu

Gao Hanrui

2001

Nguồn gốc: Comprehensive use of data structures from the round-table problem

IOI Candidate Team Papers/2001/Gao Hanrui.pdf

Abstract

Bài viết xuất phát từ một biến thể của bài toán Josephus, so sánh cách mô phỏng bằng mảng và danh sách liên kết, rồi đề xuất một cấu trúc phân đoạn kết hợp hai cách lưu trữ để cải thiện hiệu quả. Từ đó, tác giả bàn rộng hơn về việc kết hợp nhiều cấu trúc dữ liệu trong các bài có quy mô lớn.

1 Bài toán bàn tròn

Quanh một bàn tròn có $2n$ người ngồi. Trong đó n người là người tốt, n người là người xấu. Bắt đầu đếm từ người thứ nhất; khi đếm tới người thứ m , người đó lập tức bị xử tử. Sau đó bắt đầu đếm tiếp từ người ngay sau người vừa bị xử tử, rồi lại xử tử người thứ m , và cứ tiếp tục như vậy. Hỏi cần sắp xếp trước chỗ ngồi của người tốt và người xấu như thế nào để sau khi đã xử tử n người, n người còn lại trên bàn tròn đều là người tốt.

Dữ liệu vào. Mỗi dòng của tệp vào chứa hai số n và m , mô tả một bộ dữ liệu. Ràng buộc là

$$n \leq 32767, \quad m \leq 32767.$$

Dữ liệu ra. Với mỗi bộ dữ liệu, lần lượt in một lời giải. Một lời giải có thể biểu diễn bằng nhiều dòng ký tự liên tiếp, mỗi dòng không quá 50 ký tự. Trong cùng một lời giải không được có ký tự trắng và dòng trống; giữa hai lời giải kề nhau có một dòng trống. Dùng chữ cái G để biểu diễn người tốt (*good*) và B để biểu diễn người xấu (*bad*).

2 Ý tưởng chung

Ta mô phỏng quá trình thực tế, tìm vị trí của n người đầu tiên bị xử tử. Những vị trí này phải đặt người xấu; các vị trí còn lại đặt người tốt.

Ví dụ minh họa trong bản gốc dùng $n = 5, m = 3$. Quá trình loại bỏ lần lượt các vị trí trên vòng tròn $1, \dots, 10$; hình vẽ cho thấy mỗi bước xóa một điểm rồi bắt đầu đếm lại từ điểm kế tiếp. Nội dung quan trọng của hình là: bài toán cốt lõi là duy trì một vòng các phần tử còn sống, liên tục tìm phần tử thứ m kế tiếp rồi xóa nó.

3 Cách thông thường: tìm kiếm trên bảng tuyến tính

3.1 Dùng cấu trúc lưu trữ tuần tự

Dùng một mảng ghi lại vị trí ban đầu của tất cả người chưa bị xử tử. Giá trị khởi tạo là $1, 2, \dots, 2n$. Dựa vào vị trí trong mảng của người vừa bị xử tử, tức chỉ số hiện tại, ta có thể tính trực tiếp chỉ số của người tiếp theo cần bị xử tử. Sau đó xóa phần tử ấy và dịch toàn bộ các phần tử phía sau lên một vị trí.

Nếu gọi thao tác tìm người tiếp theo là **tìm điểm**, và thao tác sau khi xóa một người là **xóa điểm**, thì mảng tuần tự có ưu điểm là tìm điểm nhanh: từ chỉ số hiện tại có thể tính và định vị chính xác. Nhưng nhược điểm rõ ràng là xóa điểm chậm, vì sau mỗi lần xóa phải dịch nhiều phần tử, độ phức tạp $\mathcal{O}(n)$. Vì vậy toàn bộ chương trình có độ phức tạp $\mathcal{O}(n^2)$.

Trong ví dụ $n = 5, m = 3$, bản gốc minh họa cấu trúc tuần tự bằng các dãy còn lại sau từng lần xóa:

1 2 3 4 5 6 7 8 9 10,
1 2 4 5 6 7 8 9 10,
1 2 4 5 7 8 9 10,
1 2 4 5 7 8 10,
1 4 5 7 8 10,
1 4 5 8 10.

Các thao tác dịch trong minh họa lần lượt là 7, 4, 1, 5, 2, tổng cộng 19 lần.

3.2 Dùng cấu trúc lưu trữ liên kết

Dùng danh sách liên kết ghi lại vị trí ban đầu của mọi người chưa bị xử tử, ban đầu là $1..2n$. Mỗi khi xử tử một người, chỉ cần xóa nút tương ứng khỏi danh sách, rồi dịch con trỏ về sau $m - 1$ lần để tìm người tiếp theo.

Danh sách liên kết có ưu điểm là xóa điểm nhanh: chỉ cần sửa con trỏ của nút trước nút bị xóa. Nhược điểm là tìm điểm chậm, vì phải di chuyển con trỏ $m - 1$ lần. Do đó độ phức tạp tổng thể là $\mathcal{O}(nm)$. Với ví dụ $n = 5, m = 3$, cách ước lượng thô cho khoảng $5 \times 3 = 15$ bước di chuyển con trỏ.

Từ góc nhìn thiết kế, **tìm điểm** và **xóa điểm** là một cặp mâu thuẫn trong chương trình và cấu trúc dữ liệu. Với mảng tuần tự, tìm điểm hiệu quả nhưng xóa điểm kém; với danh sách liên kết, xóa điểm hiệu quả nhưng tìm điểm kém. Đây là hệ quả của bản thân cấu trúc dữ liệu, không thể biến mất chỉ bằng mong muốn chủ quan. Ta cần một cấu trúc sao cho cả hai thao tác đều đủ thấp về chi phí.

4 Cách cải tiến: định vị trực tiếp đã tối ưu

Ý tưởng tổng thể là vẫn giữ được khả năng **định vị trực tiếp** tương đối tốt, nhưng tránh dịch chuyển phần tử trên quy mô lớn. Vì dịch chuyển mảng nhỏ và dịch con trỏ nhỏ đều chấp nhận được, ta chia dữ liệu thành nhiều đoạn nhỏ.

Cấu trúc được thiết kế như sau. Gọi group là số đoạn lưu trữ. Mỗi đoạn giữ một mảng tuần tự nhỏ chứa các phần tử hiện còn trong đoạn đó. Ở đầu mỗi đoạn lưu một giá trị amount, biểu diễn số phần tử hiện còn của đoạn. Khi chương trình chạy, các giá trị amount giảm dần.

Ví dụ slide gốc mô tả dãy 1, 2, ..., 10 được chia thành 4 đoạn:

đoạn	amount	phần tử
1	3	1, 2, 3
2	3	4, 5, 6
3	3	7, 8, 9
4	1	10

Trong minh họa, tổng số thao tác là $1 + 2 + 2 + 3 + 5 = 13$, ít hơn cách tuần tự thuần túy.

Cấu trúc phân đoạn này có thể xem là sản phẩm kết hợp giữa lưu trữ liên kết và lưu trữ tuần tự. Giữa các đoạn, ta dùng tư tưởng của danh sách liên kết hoặc ít nhất là quản lý theo nhóm để bỏ qua những đoạn rỗng hay ít phần tử; bên trong mỗi đoạn, ta dùng mảng tuần tự để định vị và xóa trên quy mô nhỏ. Nhờ vậy nó kế thừa ưu điểm của hai “cấu trúc cha”, đồng thời tránh phần lớn khuyết điểm của chúng.

Phương pháp này thể hiện khá tốt tư tưởng định vị trực tiếp. Sau khi chia các nút thành nhiều đoạn, mỗi lần xóa một nút chỉ phải dịch chuyển một số phần tử tương đối ít trong đoạn, nên hiệu quả chương trình tăng rõ rệt. Giá trị m càng lớn, hiệu quả cải tiến càng thấy rõ.

5 So sánh hiệu quả

Bảng sau được trích lại từ slide gốc. Máy thử nghiệm là P166; phương pháp cải tiến dùng $\text{amount}=400$.

Dữ liệu	Mảng tìm kiếm	Định vị tối ưu	Tỷ lệ
$n = 200, m = 100$	0.000s	0.000s	–
$n = 1000, m = 50$	0.440s	0.000s	–
$n = 32767, m = 200$	5.870s	0.930s	6.312
$n = 32767, m = 1000$	29.440s	0.980s	30.041
$n = 32767, m = 10000$	294.120s	1.260s	233.43
$n = 32767, m = 20000$	588.530s	1.590s	370.14
$n = 32767, m = 32767$	963.560s	1.970s	489.12

Bảng cho thấy khi quy mô lớn và m lớn, cấu trúc kết hợp vượt xa cách dùng mảng tuyến tính đơn thuần.

6 Mở rộng

Mở rộng ngang. Bài toán bàn tròn thuộc họ bài Josephus. Các bài tương tự gồm *trò chơi lật bài, khi chọn vua*, v.v. Khi nhận ra cấu trúc Josephus, ta có thể tái sử dụng cách mô phỏng vòng tròn và cách chọn cấu trúc dữ liệu phù hợp.

Mở rộng dọc. Trong các bài có quy mô dữ liệu lớn, việc kết hợp cấu trúc dữ liệu thường rất hữu ích. Bản gốc nêu bài *Hidden Code* của IOI 1999: nếu xây dựng được một cấu trúc kết hợp giữa liên kết và tuần tự, hiệu quả chương trình sẽ cao. Ví dụ slide gốc minh họa sâu

CBDADCCBADCA

được tổ chức bằng cấu trúc vừa có liên kết giữa các khối, vừa có mảng tuần tự trong từng khối.

Cấu trúc kết hợp kiểu này đơn giản, hiệu quả rõ rệt, và ứng dụng khá rộng. Ngoài việc kết hợp liên kết với tuần tự, còn có các dạng kết hợp khác, chẳng hạn ánh xạ một-một giữa heap nhị phân và cấu trúc tuần tự. Trong một số bài toán, các kết hợp như vậy cho hiệu quả rất tốt.

7 Kết luận

Xu Zhou từng viết trong bài *Bàn về phương thức tư duy dạng mạng và ứng dụng của nó*: “cốt lõi của tư duy dạng mạng là liên hệ”. Khi làm bài, ta nên đào sâu mối liên hệ giữa điều kiện đề bài, các cấu trúc dữ liệu và thuật toán. Làm được như vậy, ta mới giải quyết bài tốt hơn và nâng cao năng lực của mình.

Bài viết này chỉ xuất phát từ một lớp bài rất thường gặp: bài Josephus, hay hoán vị Josephus, rồi từ đó dẫn tới việc kết hợp cấu trúc dữ liệu. Đối với các bài tin học có hình thức đa dạng, việc kết hợp cấu trúc dữ liệu chỉ là một khía cạnh nhỏ của chiến lược giải bài. Nhưng nếu với mỗi bài toán, thuật toán và cấu trúc dữ liệu, ta đều đào sâu liên hệ của nó với những thứ khác, thì tự nhiên ta sẽ xây dựng được một mạng tri thức và biết cách tổng hợp khi cần. Điều này rất có ích cho học tập và nghiên cứu.

Cuối cùng cần nhấn mạnh rằng “kết hợp cấu trúc dữ liệu” ở đây không chỉ là kết hợp về hình thức, mà quan trọng hơn là kết hợp về tư tưởng, tức nội hàm. Chỉ cần trong tư tưởng đã thể hiện ưu điểm của hai hay nhiều cấu trúc dữ liệu, và trong thao tác đã phát huy được các ưu điểm ấy, thì về căn bản ta đã đạt được mục đích của sự kết hợp.